

Architecture

Cohort 1 Group 7

Group Members:

Kyle Clifton (wnp512)

Bailey Evers (rkh552)

Matt Hollyhead (fpk506)

William Martin (zjc524)

Max Pither (rkm538)

Enkhtuguldur Sarantsetseg (wbj512)

Ronny Watt (dvn513)

Introduction:

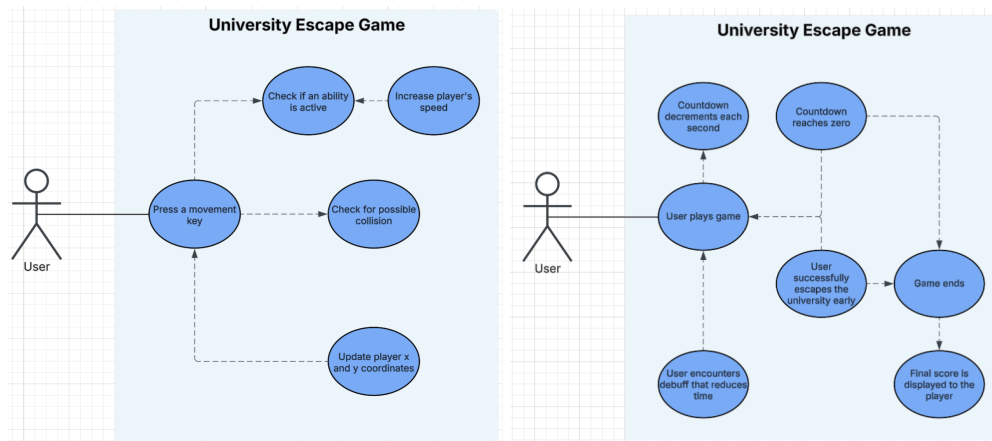
This section outlines the architecture of our maze-based puzzle video game, showing how its main systems work together to create our game. The diagrams highlight how the player, the environment, and the supporting systems (such as events, scoring, and user interface) interact to form the core game loop.

We used UML 2.5 to model the system because it provides a clear and standard way to represent both the structure and behaviour of object-oriented designs. All diagrams were created in LucidChart, which allowed our team to collaborate easily and keep our designs consistent as the project evolved. The sections that follow include both structural diagrams, showing how the main components fit together, and behavioural diagrams, illustrating how those components interact during gameplay. Together, they provide a clear picture of how the game is organised and how players will experience it.

Architecture design

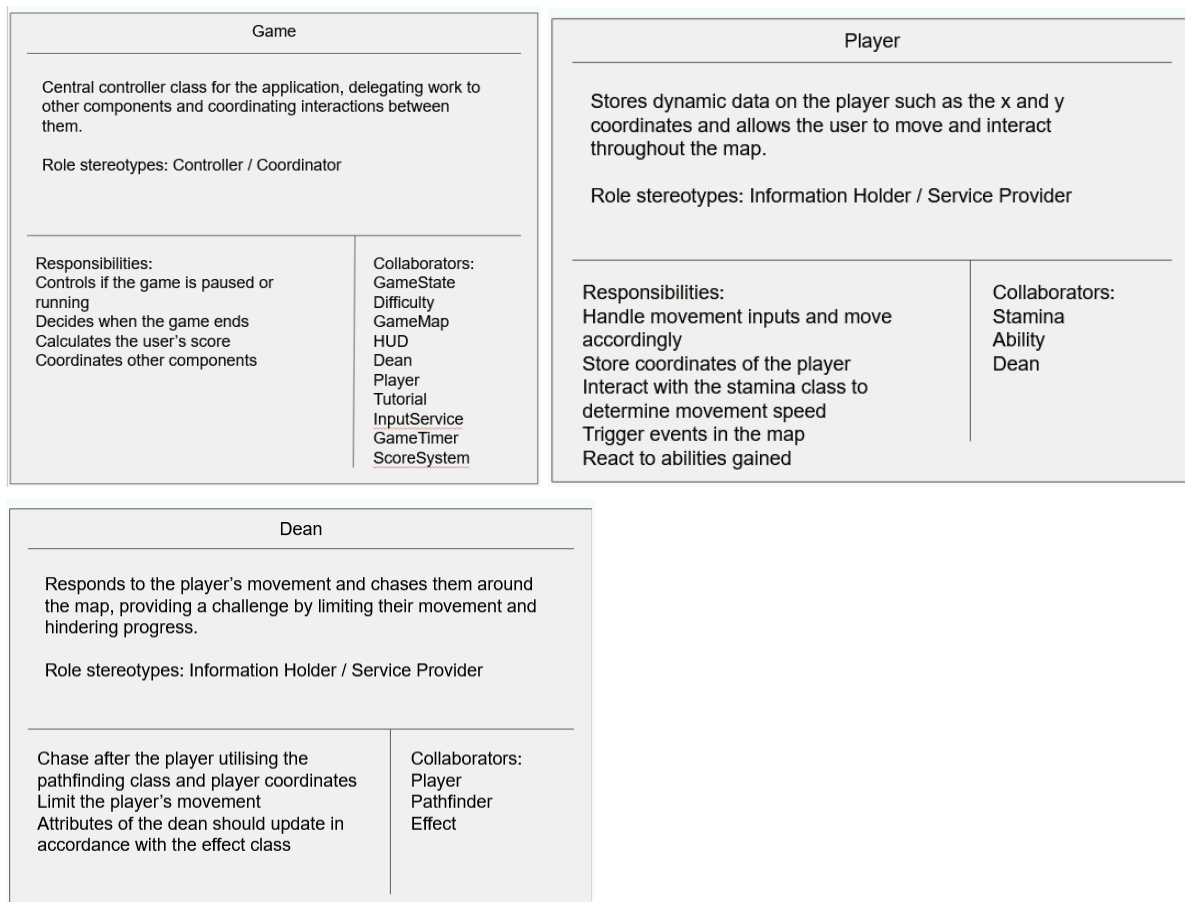
We made use of use case diagrams as the first step in designing architecture, since they would highlight some of the key aspects required for the game and give us an initial idea of how different components would need to interact with each other to create a functional application. By modelling how these scenarios could play out, we were provided with a starting point to further develop the architecture.

Name	User inputs movement	Player Finishes Game
Primary Actor	User	User
Trigger	User presses a W/A/S/D key during gameplay	User trigger game-ending condition
Main Success Scenario	· Player class checks if player has “ability” which affects movement · Player class checks if there would be a collision with obstacle such as wall or Dean	· Countdown reaches 0
Secondary Scenario	· Obstacle is in the player’s way ○ Player cannot move in that direction	· Player escapes university early
Success Postcondition	Player’s x and y coordinates are updated accordingly	Game ends and final score is displayed to player
Minimal Postcondition	-	Game continues as countdown decrements



Candidate responsibility cards

These are the finalised candidate responsibility cards, representing the three main classes that will be used in the program architecture, ready to be mapped into a UML class diagram and implemented. The centralised design around the game class should allow for any additional features to be easily added modularly, which is well suited for the prototype, and the collaborators that communicate with each of these classes are suitably decomposed into necessary chunks so that development and maintenance should also be relatively simple and easy.



UML Class Diagram

Requirements:

Req1 — The game shall be singleplayer only. **Req2** — The game shall have a maximum duration of 5 minutes. **Req3** — The game may end early if the player reaches an exit. **Req4** — The Dean shall chase the player. **Req5** — The Dean shall limit the player's movement but cannot end the game. **Req6** — There shall be at least 5 visible negative events that disadvantage the player. **Req7** — There shall be at least 5 visible positive events that advantage the player. **Req8** — There shall be at least 3 hidden events that may advantage or disadvantage the player. **Req9** — The player shall be able to escape via an exit. **Req10** — The player shall be able to pause the game, which halts gameplay and the timer. **Req11** — The game shall calculate a score based on elapsed time and triggered events. **Req12** — The game shall be playable across major operating systems. **Req13** — The game shall include an introductory tutorial that teaches controls and objectives. **Req14** — The game shall use one large continuous map. **Req15** — The game shall use a top-down camera perspective. **Req16** — Events may grant abilities to the player. **Req17** — The player shall have a stamina bar that limits movement speed. **Req18** — The game may include multiple difficulty modes that affect gameplay parameters.

Description:

Overview

This UML class diagram represents a single-player puzzle game focused on exploration, challenges, and time-based objectives. It highlights the key systems that shape gameplay — from the player and antagonist to events, scoring, and UI — showing how they interact to create a cohesive experience.

Core Classes

The Game class acts as the central controller, managing game flow, state, timer, difficulty, and score. It coordinates interactions between the player, the Dean, and the world.

GameState defines main phases like *Intro*, *Running*, *Paused*, and *Ended*, ensuring smooth transitions, while Difficulty adjusts gameplay intensity by influencing factors such as event frequency or stamina loss.

Actor Classes

The Player represents the user-controlled character, capable of movement, triggering events, and escaping through exits. It has stamina and can gain temporary boosts from positive events. The Dean serves as the pursuing antagonist — applying slowing effects and maintaining tension without directly ending the game.

World Classes

The GameMap defines the playable area, including its layout, spawn points, and event locations. Exit objects act as escape points that allow early completion, providing alternate win conditions.

Events System

The Event class covers all in-game occurrences that help or hinder the player, classified by their *Impact* — *Advantage* or *Disadvantage* — and visibility. The EventFactory controls event creation, ensuring a balanced mix of positive, negative, and hidden events across the map.

User Interface

The HUD displays essential information such as score, timer, and stamina, and includes the PauseMenu for pausing gameplay. The Tutorial introduces players to controls, objectives, and core mechanics before the game begins.

Infrastructure and Helpers

The RenderService handles visual output through the UI, while the Pathfinder assists the Dean with navigation. The ScoreSystem calculates points based on time and triggered events, rewarding both speed and exploration.

Relationships and Patterns

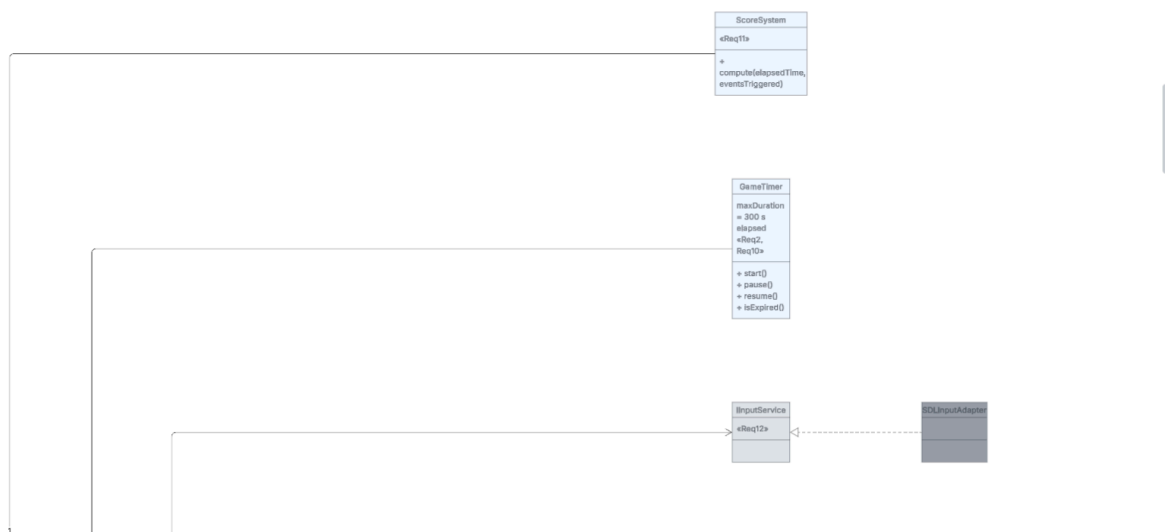
Composition indicates ownership (e.g., the *Game* owns its *Player*), while aggregation represents shared links such as a *Player* having multiple *Abilities*. Multiplicity values (1, 0..*, 1..1) specify how many instances can relate, clarifying the system's structure and constraints.

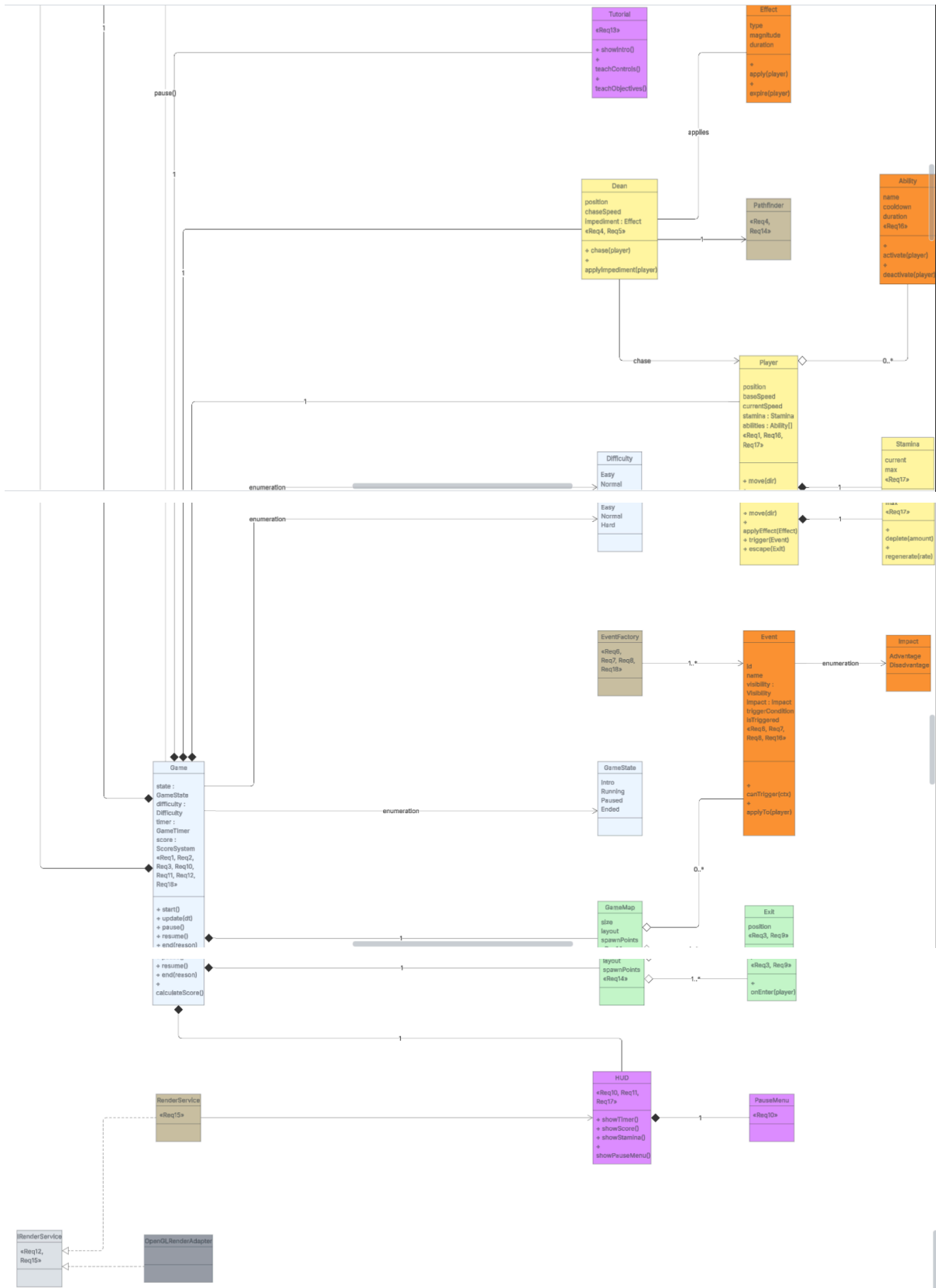
Requirements and Design Links

Each «ReqX» label connects to a specific design requirement. For example, *Req1* ensures single-player mode, while *Req4* defines the Dean's chasing behaviour. These links tie the architecture directly to the game's functional goals.

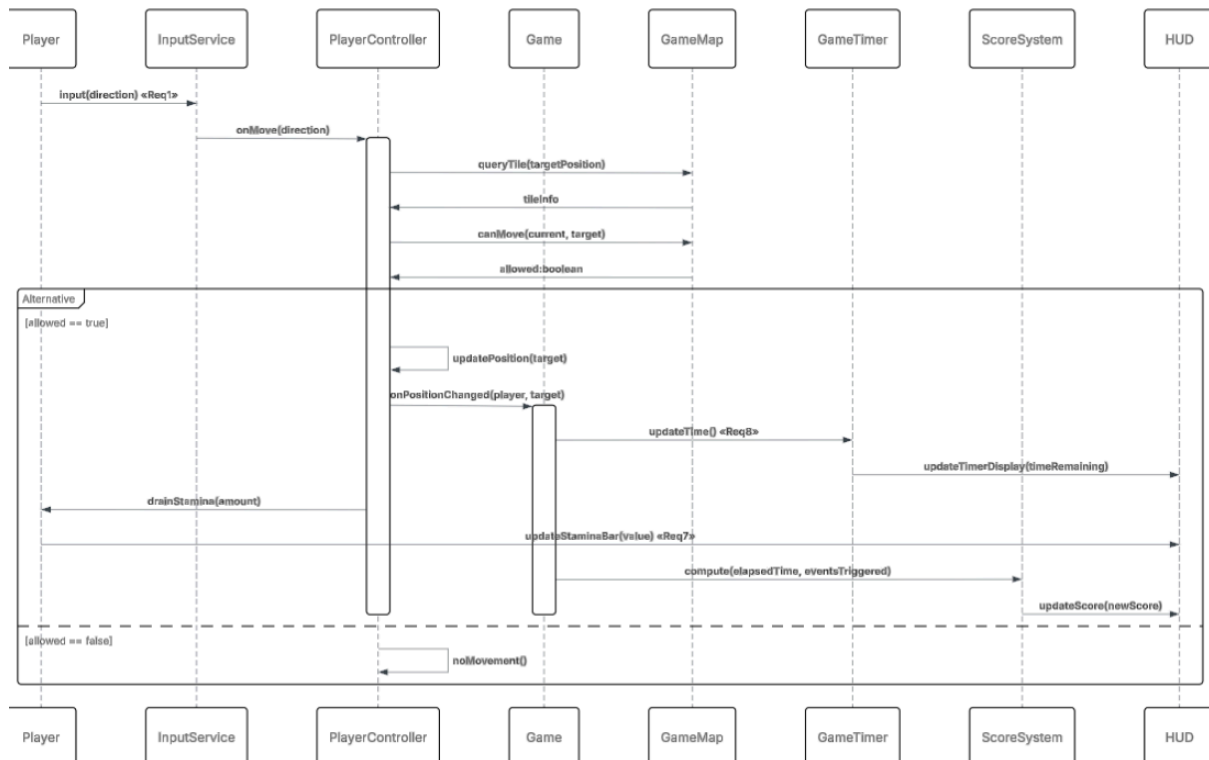
Summary

The architecture centres on the Game class, which orchestrates all major systems. The player explores, triggers events, and avoids the Dean, while the HUD, tutorial, and scoring systems support a clear, engaging gameplay loop. Together, these components deliver a structured, adaptable, and immersive single-player experience.





Player Movement (Sequence Diagram)



This sequence diagram shows how the game processes a player's movement during gameplay. When the Player inputs a direction, the InputService passes that command to the PlayerController, which checks the GameMap to see if the move is possible. If the space ahead is clear, the player's position is updated, and the Game coordinates changes across several systems — updating the GameTimer, adjusting stamina, and recalculating the score. These updates are then shown on the HUD, refreshing the timer (Req8), stamina bar (Req7), and score display in real time to keep the player informed.

The alt fragment highlights two possible outcomes: if the movement is allowed, all related updates occur; if not, no action is taken. This behaviour supports key design requirements such as Req1 (handling player input), Req7 (stamina display), Req8 (time management), and Req11 (scoring). Together, these interactions ensure the movement system feels responsive, consistent, and clearly communicated to the player.